

信息学奥林匹克竞赛

Olympiad in Informatics

动态规划之区间DP

陈 真

上海市实验学校

【题目01】山区建小学

【问题描述】 政府在某山区修建了一条道路，恰好穿越总共 m 个村庄的每个村庄一次，没有回路或交叉，任意两个村庄只能通过这条路来往。已知任意两个相邻的村庄之间的距离为 d_i （为正整数），其中， $0 < i < m$ 。为了提高山区的文化素质，政府又决定从 m 个村中选择 n 个村建小学（设 $0 < n \leq m < 500$ ）。请根据给定的 m 、 n 以及所有相邻村庄的距离，选择在哪些村庄建小学，才使得所有村到最近小学的距离总和最小，计算最小值。

【输入格式】 第1行为 m 和 n ，其间用空格间隔

第2行为 $(m-1)$ 个整数，依次表示从一端到另一端的相邻村庄的距离，整数之间以空格间隔。

例如：

10 3

2 4 6 5 2 4 3 1 3

表示在10个村庄建3所学校。第1个村庄与第2个村庄距离为2，第2个村庄与第3个村庄距离为4，第3个村庄与第4个村庄距离为6，...，第9个村庄到第10个村庄的距离为3。

【输出格式】 各村庄到最近学校的距离之和的最小值。

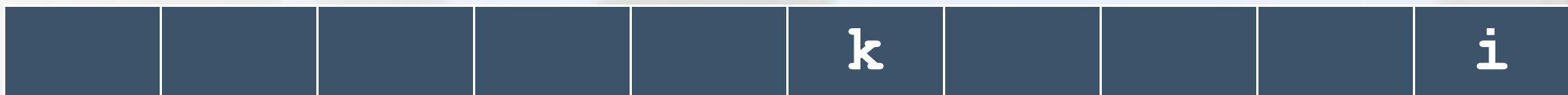
【样例输入】 10 2 3 1 3 1 1 1 1 1 3

【样例输出】 18

经典例题

【题目01】山区建小学

假设 $DP[i][j]$ 表示前 i 个村庄建 j 个小学的最小花费



$$DP[i][j] = \min\{DP[i][j], DP[k][j-1] + (k+1 \text{ 到 } i \text{ 建立一个学校的花费})\}$$

区间动态规划

【问题02】石子合并（直线型）

【题目描述】在操场上沿一直线排列着 n 堆石子。现要将石子有次序地合并成一堆。规定每次只能选相邻的两堆石子合并成新的一堆，并将新的一堆石子数计为该次合并的得分。我们希望这 $n-1$ 次合并后得到的得分总和最小。

【输入格式】

第一行有一个正整数 n ($n \leq 300$)，表示石子的堆数；

第二行有 n 个正整数，表示每一堆石子的石子数，每两个数之间用一个空格隔开。它们都不大于10000。

【输出格式】一行，一个整数，表示答案。

【输入数据】

3
1 2 9

【输出数据】

15

区间动态规划

【问题02】石子合并（直线型）

◆ 设 $f[i][j]$ 表示从区间 i 到 j 合并产生的最小值

◆ 对于区间 $[L, R]$ 进行拆分

$$f[L][R] = \{f[L][R], f[L][k] + f[k+1][R] + \text{代价}\}$$

◆ 枚举区间长度来确定不同的 $[L, R]$ 的区间最值。区间长度 $1 \rightarrow n$

◆ 最终答案为 $f[1][N]$

区间动态规划

区间动态规划问题一般都是考虑，对于每个区间，他们的**最优值都是由几段更小区间的最优值得到**，分而治之的思想，将一个区间问题不断划分为更小区间直至一个元素组成的区间，枚举他们的组合，求**合并后的最优值**。所以，**区间DP是阶段以区间长度划分的一类DP问题**。最明显的特点是**大区间是由小区间推导出来的**。

区间DP的状态设定，**一般**设定为 $f[i][j]$ 表示从第 i 个元素到第 j 个元素这个区间的**最优值**什么？

区间DP的本质就是对区间信息的合并（这类似于树形DP的合并）。

在区间DP中，我们可以通过对区间信息的合并，完成对一系列问题最优解/最小花费的求值。

套路：枚举区间长度，枚举左端点，求出右端点，枚举断点

区间动态规划

阶段划分

区间DP常常以**区间为单位进行阶段划分**。“合并石子”中，由于每次必须合并相邻两堆石子，合并的过程将n堆划分成了若干堆，每一堆均为一段连续的区间。因此可按区间长度从小到大划分阶段，设阶段变量len表示区间长度为len。

设计状态

对于阶段len，状态即这一阶段包含的可能的区间信息，设 $f[len][i][j]$ 代表将区间长度为len，区间为 $[i,j]$ 的石子合并成一堆的最优值。因为 $j-i+1=len$ ，状态变量len、i、j可互相导出，所以状态变量len、i、j可省略一维，**设 $f[i][j]$ 代表将区间 $[i,j]$ 的石子合并成一堆的最优值**。

确定决策

对于状态 $f[i][j]$ ，由于每次只能合并相邻两堆，**决策即将区间 $[i,j]$ 拆分为两个区间的断点**，设决策变量k代表上一阶段合并两堆石子的区间为 $[i,k]$ 和 $[k+1][j]$ 。

转移方程

$$f[i][j] = \min(f[i][k] + f[k+1][j] + \text{sum}[j] - \text{sum}[i-1]) \quad (i \leq k < j)$$
，sum[j]为前j堆石子的重量， $\text{sum}[j] - \text{sum}[i-1]$ 为区间 $[i,j]$ 的石子的重量。

确定目标

边界为 $f[i][i] = 0$ ，其余 $f[i][j] = +\infty$ ，即阶段1（区间长度为1）中，将区间 $[i,i]$ 的石子合并的价值为0。目标解为 $f[1][n]$ ，即将区间 $[1,n]$ 的石子合并的最优价值。

区间动态规划

$f[i][j]$ 代表将区间 $[i,j]$ 的石子合并成一堆的最小消耗体力

```
2  int f[N][N];
3  memset(f, 0x3f, sizeof(f));
4  for(int i=1; i<=n; i++) f[i][i]=0;
5  for(int len=2; len<=n; len++){//枚举阶段
6      for(int i=1; i<=n-len+1; i++){//枚举状态
7          int j=i+len-1;
8          for(int k=i; k<j; k++){//枚举决策
9              f[i][j]=min(f[i][j], f[i][k]+f[k+1][j]+sum[j]-sum[i-1]);
10             }
11         }
12     }
```

【温馨提示】在书写代码时，建议遵循**枚举阶段**、**枚举状态**、**枚举决策**这一顺序，确保计算第 len 阶段的状态时，之前阶段的所有状态均已求解。

区间动态规划

【题目03】石子合并 (NOI1995)

【题目描述】 在一个圆形操场的四周摆放 N 堆石子,现要将石子有次序地合并成一堆.规定每次只能选相邻的2堆合并成新的一堆,并将新的一堆的石子数,记为该次合并的得分。
试设计出一个算法,计算出将 N 堆石子合并成 1 堆的最小得分和最大得分。

【输入格式】 数据的第 1 行是正整数 N , 表示有 N 堆石子。

第 2 行有 N 个整数, 第 i 个整数 a_i 表示第 i 堆石子的个数。

【输出格式】 输出共 2 行, 第 1 行为最小得分, 第 2 行为最大得分。

【输入数据】

4
4 5 9 4

【输出数据】

43
54

区间动态规划

【题目03】石子合并 (NOI1995)

◆环形处理：断环成链

```
for(int i=1;i<=n;i++){  
    cin>>w[i];  
    w[i+n]=w[i];  
}
```

◆借鉴直线型求最大值与最小值

◆答案的处理？

```
for(int i=1;i<=n;i++){  
    ans1=min(ans1,dp1[i][i+n-1]);  
    ans2=max(ans2,dp2[i][i+n-1]);  
}
```

区间动态规划

【题目03】 [USACO16OPEN] 248 G

【题目描述】 贝西喜欢在手机上下载游戏来玩，尽管她确实觉得对于自己巨大的蹄子来说，小小的触摸屏用起来相当笨拙。

她对当前正在玩的这个游戏特别感兴趣。游戏开始时给定一个包含 N 个正整数的序列 ($2 \leq N \leq 48$)，每个数的范围在 $1..40$ 之间。在一次操作中，贝西可以选择**两个相邻且相等的数**，将它们替换为一个比原数大 1 的数（例如，她可以将两个相邻的 7 替换为一个 8 ）。游戏的目标是最大化最终序列中的最大数值。请帮助贝西获得尽可能高的分数！

【输入格式】 第一行输入包含 N ，接下来的 N 行给出游戏开始时序列的 N 个数字。

【输出格式】 请输出贝西能生成的最大整数。

【输入样例】

4
1
1
1
1
2

【输出样例】

3

【解析】

动态转移方程: $dp[l][r] = \max\{dp[l][r], dp[l][k] + 1\}$

思考： 设置断点的前提条件是？

进一步思考： N 的数据范围继续扩大 ($2 \leq N \leq 2^{18}$) 又该如何求解？

区间动态规划

【题目03】 [USACO16OPEN] 248 G

【解析】进一步思考：N的数据范围继续扩大（ $2 \leq N \leq 2^{18}$ ）又该如何求解？

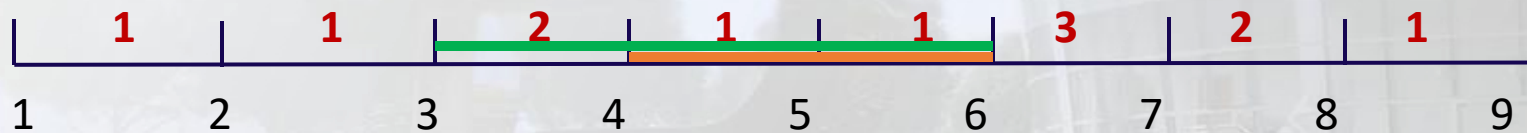
设 $dp[i][k]$ 表示第一个数合并得到数值为 k ，能够向右拓展的最右断点，对数据 $num[i]$

初始值为： $dp[i][num[i]] = i + 1$

例如数据：8个数，分别1, 1, 2, 1, 1, 3, 2, 1

初始值 $dp[1][1]=2, dp[2][1]=3, dp[3][2]=4, dp[4][1]=5,$

$dp[5][1]=6, dp[6][3]=7, dp[7][2]=8, dp[8][1]=9$



举例： $dp[4][2] = dp[dp[5][1]][1] = 6$

$dp[3][3] = dp[dp[3][2]][2] = dp[4][2] = 6$

动态转移方程： $dp[i][k] = dp[dp[i][k-1]][k-1]$

如果 $dp[i][k] \neq 0$,说明数值 k 是可以合并得到的。

$ans = \max(ans, k)$

k 是阶段， $1 \sim n$ 是决策，思考 k 的最大值，最小值是？

区间DP应用技巧

应用技巧1：依据问题特性，辨识区间DP。

线性DP和区间DP往往在实际求解中会混淆，区别是否为区间DP的重点在于辨识问题是否具有“区间”特性，“**区间**”特性常常表现为：**操作的对象是一段区间、操作剩下的对象是一段区间**等等。例如合并石子问题的“区间”特性就在于每次操作的对象（合并的两堆石子）是一段区间。此外，区间DP的程序实现复杂度一般比较高，通过观察问题规模及数据范围，也可得到一些提示。

区间DP应用技巧

应用技巧1：依据问题特性，辨识区间DP。

【例题04】矩阵取数游戏（题目来源：NOIP2007）

【问题描述】帅帅经常跟同学玩一个矩阵取数游戏：对于一个给定的 $n \times m$ 的矩阵，矩阵中的每个元素 a_{ij} 均为非负数。游戏规则如下：

1. 每次取数时从每行各取走一个元素，共 n 个。 m 次后取完矩阵所有元素；
2. 每次取走的各个元素只能是该元素所在行的行首或者行尾；
3. 每次取数都有一个得分值，为每行取数的得分之和，每行取数的得分=被取走的元素值 $\times 2^i$ ，其中 i 表示第 i 次取数（从1开始编号）；
4. 游戏结束总得分为 m 次取数得分之和。

帅帅想请你帮忙写一个程序，对于任意矩阵，可以求出取数后的最大得分。

【输入格式】输入包括 $n+1$ 行：

第1行为两个用空格隔开的整数 n 和 m ；

第2~ $n+1$ 行为 $n \times m$ 矩阵，其中每行有 m 个用单个空格隔开的非负整数。

【输出格式】仅包含1行，为一个整数，即输入矩阵取数后的最大得分。

样例输入	样例输出
2 3 1 2 3 3 4 2	82

区间DP应用技巧

应用技巧1：依据问题特性，辨识区间DP。

【例题04】矩阵取数游戏（题目来源：NOIP2007）

由于每次取数只能取行首或行尾，那么本问题的“区间”特性体现在：每次操作后剩下的是一段区间，因此可以从区间DP的角度分析问题。

设 $f[i][j]$ 代表一行取完数后剩下区间为 $[i,j]$ 时得到的最大得分

对于当前阶段，决策即选择行首还是行尾的数，

若选择行首 $a[i]$ ，则 $f[i+1][j]=f[i][j]+a[i] \cdot 2^{(n-j+i)}$ ，

若选择行尾的数 $a[j]$ ，则 $f[i][j-1]=f[i][j]+a[j] \cdot 2^{(n-j+i)}$ ；

边界为 $f[1][n]=0$ ，即第一行取完数后剩下区间为 $[1,n]$ 的数的最大得分

目标解为 $\max(f[i][i]+2^n) \quad (1 \leq i \leq n)$ 剩下区间为 $[i,i]$ 的得到的最大得分加上取剩下的第 i 个数的得分之和

区间DP应用技巧

应用技巧1：依据问题特性，辨识区间DP。

【例题04】矩阵取数游戏（题目来源：NOIP2007）

//f[i][j]代表一行取完数后剩下区间
为[i,j]时得到的最大得分。

```
for(int p=1;p<=n;p++){
    for(int q=1;q<=m;q++)cin>>a[q];
    memset(f,0,sizeof(f));
    for(int len=m;len>1;len--){//枚举阶段
        for(int i=1;i+len-1<=m;i++){//枚举状态
            int j=i+len-1;
            //枚举决策
            //决策1: 取行首
            f[i+1][j]=max(f[i+1][j],f[i][j]+a[i]*cal(m-j+i));
            //决策2: 取行尾
            f[i][j-1]=max(f[i][j-1],f[i][j]+a[j]*cal(m-j+i));
        }
    }
    __int128 ans1=0;
    for(int i=1;i<=m;i++){
        ans1=max(ans1,f[i][i]+a[i]*cal(m));
    }
    ans+=ans1;
}
```


区间DP应用技巧

应用技巧2：断环成链，巧解环形DP。

【题目05】能量项链 (NOIP2006)

【问题描述】例如：设 $N=4$ ，4颗珠子的头标记与尾标记依次(2,3)(3,5)(5,10)(10,2)。我们用记号 $\oplus$ 表示两颗珠子的聚合操作， $(j\oplus k)$ 表示第 j,k 两颗珠子聚合后所释放的能量。则第4、1两颗珠子聚合后释放的能量为：

$$(4\oplus 1)=10\times 2\times 3=60。$$

这一串项链可以得到最优值的一个聚合顺序所释放的总能量为：

$$\begin{array}{ccccccc} & & 4 & & & & \\ & & 2 & 3 & 5 & 10 & \\ & & & & & & 710 \end{array}$$

$$((4\oplus 1)\oplus 2)\oplus 3 = 10\times 2\times 3 + 10\times 3\times 5 + 10\times 5\times 10 = 710$$

【输入格式】第一行是一个正整数 $N(4\leq N\leq 100)$ ，表示项链上珠子的个数。

第二行是 N 个用空格隔开的正整数，所有的数均不超过1000。第 i 个数为第 i 颗珠子的头标记($1\leq i\leq N$)，当 $i<N$ 时，第 i 颗珠子的尾标记应该等于第 $i+1$ 颗珠子的头标记。第 N 颗珠子的尾标记应该等于第1颗珠子的头标记。

至于珠子的顺序，你可以这样确定：将项链放到桌面上，不要出现交叉，随意指定第一颗珠子，然后按顺时针方向确定其他珠子的顺序。

【输出格式】一个正整数 $E(E\leq 2.1\times 10^9)$ ，为一个最优聚合顺序所释放的总能量。

区间DP应用技巧

应用技巧2：断环成链，巧解环形DP。

【题目05】能量项链 (NOIP2006)

◆ 环形问题：断环成链

◆ 设 $f[L][R]$ 为？

$$f[L][R] = \max \{ f[L][R], f[L][k] + f[k][R] + e[L] * e[R] * e[k] \}$$

◆ 枚举区间长度，枚举断点

◆ 获得答案

区间DP应用技巧

应用技巧3：增设DP状态，记录关键因素。

常见区间DP问题求解时设置的状态为 $f[i][j]$ ，其中 i 和 j 常表示区间的两个端点。然而区间DP问题的状态有时不仅仅只有两个维度，若状态 $f[i][j]$ 不足以表示状态，则需根据问题需求增加状态维度。增加状态维度除了像多维DP一样考虑某阶段的客观条件以外，有时也需要从区间合并的角度出发去考虑增加状态维度。

区间DP应用技巧

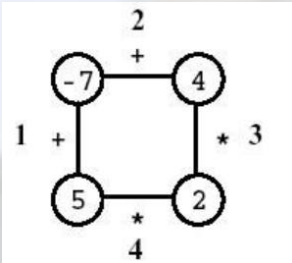
应用技巧3：增设DP状态，记录关键因素。

【例题06】Polygon（题目来源：IOI1998）

【问题描述】 一个玩家在一个有n个顶点的多边形上的游戏，如图所示，其中 $n = 4$ 。每个顶点用整数标记，每个边用符号+（加）或符号*（乘积）标记。

第一步，删除其中一条边。随后每一步：选择一条边连接的两个顶点V1和V2，用边上的运算符计算V1和V2得到的结果来替换这两个顶点。游戏结束时，只有一个顶点，没有多余的边。如下图所示，玩家先移除编号为3的边；之后，玩家选择计算编号为1的边，然后计算编号为4的边，最后，计算编号为2的边。结果是0。

编写一个程序，给定一个多边形，计算最高可能的分数。

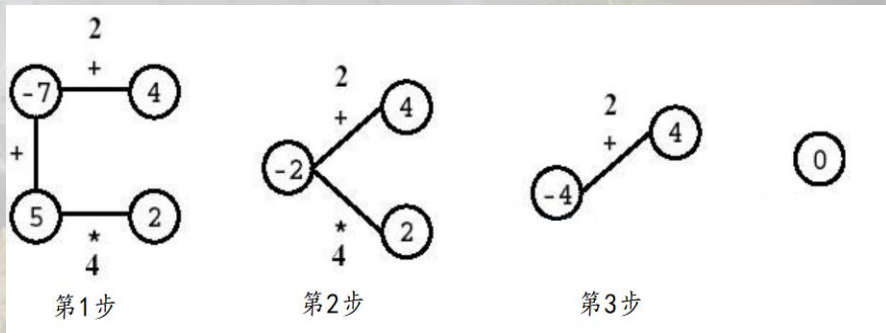


【输入格式】 第一行有一个整数n，为多边形顶点个数与边数。

第二行有2n个输入，每两个输入中第一个是字符，第二个是数字。其中第一个字符为第1条边的计算符号(t代表相加，x代表相乘)，第二个代表顶点上的数字。顶点首尾相连。

【输出格式】 第1行，输出最高的分数。

第2行，写出所有第一步可以删除的边的编号，使得删除该边后能得到相同的最高分，边的编号必须严格递增。



区间DP应用技巧

应用技巧3：增设DP状态，记录关键因素。

样例输入	样例输出
4	33
t -7 t 4 x 2 x 5	1 2

【例题06】 Polygon （题目来源：IOI1998）

区间DP应用技巧

应用技巧3：增设DP状态，记录关键因素。

【例题06】Polygon（题目来源：IOI1998）

【问题描述】 一个玩家在一个有 n 个顶点的多边形上的游戏，如图所示，其中 $n = 4$ 。每个顶点用整数标记，每个边用符号+（加）或符号*（乘积）标记。

第一步，删除其中一条边。随后每一步：选择一条边连接的两个顶点 $V1$ 和 $V2$ ，用边上的运算符计算 $V1$ 和 $V2$ 得到的结果来替换这两个顶点。游戏结束时，只有一个顶点，没有多余的边。如下图所示，玩家先移除编号为3的边；之后，玩家选择计算编号为1的边，然后计算编号为4的边，最后，计算编号为2的边。结果是0。

编写一个程序，给定一个多边形，计算最高可能的分数。

样例输入	样例输出
4	33
t -7 t 4 x 2 x 5	1 2

区间DP应用技巧

应用技巧3：增设DP状态，记录关键因素。

【例题06】Polygon（题目来源：IOI1998）

对于环形问题，可通过复制链解决。考虑加入乘法操作后，区间 $[i,j]$ 的最高分不一定等于“区间 $[i,k]$ 的最高分 op 区间 $[k+1,j]$ 的最高分”（ op 代表连接顶点 k 和 $k+1$ 的边上的运算符），对于乘法而言，两个区间的最低分（为负数时）相乘可能得到区间 $[i,j]$ 的最高分。

区间 $[i,j]$ 的最优解由子区间 $[i,k]$ 、 $[k+1,j]$ 的最优解进行合并时，我们除了需要知道子区间 $[i,k]$ 、 $[k+1,j]$ 的最高分以外，还需要知道子区间 $[i,k]$ 、 $[k+1,j]$ 的最低分。

设 $f[i][j][0/1]$ 代表区间 $[i,j]$ 的数字合并得到的最高/最低分，那么：

1.若 $\text{op} = '+'$ ： $f[i][j][0] = f[i][k][0] \text{ op } f[k+1][j][0]$ ， $f[i][j][1] = f[i][k][1] \text{ op } f[k+1][j][1]$ 。

2.若 $\text{op} = '*'$ ： $f[i][j][0] = \max(f[i][k][0] \text{ op } f[k+1][j][0], f[i][k][1] \text{ op } f[k+1][j][1])$

$f[i][j][1] = \min(f[i][k][1] \text{ op } f[k+1][j][1], f[i][k][0] \text{ op } f[k+1][j][1], f[i][k][1] \text{ op } f[k+1][j][0])$ 。

边界为 $f[i][j][0] = -\infty$ ， $f[i][j][1] = +\infty$ ， $f[i][i][0] = f[i][i][1] = a[i]$ ， $a[i]$ 为顶点 i 的数字。

目标解为 $\max(f[i][i+n-1][0]), 1 \leq i < n$ 。

区间DP应用技巧

应用技巧3：增设DP状态，记录关键因素。

【例题06】Polygon（题目来源：IOI1998）

```
for(int i=1;i<=2*n;i++){
    f[i][i][0]=f[i][i][1]=a[i];
}
for(int len=2;len<=n;len++){
    for(int i=1;i+len-1<=2*n;i++){
        int j=i+len-1;
        for(int k=i;k<j;k++){
            if(op[k]==0){//+
                f[i][j][0]=max(f[i][j][0],f[i][k][0]+f[k+1][j][0]);
                f[i][j][1]=min(f[i][j][1],f[i][k][1]+f[k+1][j][1]);
            }
            else{/*
                f[i][j][0]=max(f[i][j][0],max(f[i][k][0]*f[k+1][j][0],f[i][k][1]*f[k+1][j][1]));
                f[i][j][1]=min(f[i][j][1],min(f[i][k][1]*f[k+1][j][1],min(f[i][k][1]*f[k+1][j][0],f[i][k][0]*f[k+1][j][1])));
            }
        }
    }
}
```

//f[i][j][0/1]代表区间[i,j]的数字合并得到的最高/最低分

```
int ans=-1e9;
for(int i=1;i<=n;i++){
    ans=max(ans,f[i][i+n-1][0]);
}
```


区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

【题目07】关灯

在某些区间DP问题中，当前决策可能会影响未来行动的费用。若当前决策对未来的影响只与当前决策有关，则可将对未来费用的影响算作当前决策的费用；若当前决策对未来的影响不仅与当前决策有关，还与未来情况有关，则可新增状态假设未来情况，待到未来决策时使用假设的状态。

【问题描述】 Mr.Chen得到了一份有趣而高薪的工作。每天早晨她必须关掉她所在村庄的街灯。所有的街灯都被设置在一条直路的同一侧。 Mr.Chen每晚到早晨5点钟都在晚会上，然后她开始关灯。开始时，她站在某一盏路灯的旁边。每盏灯都有一个给定功率的电灯泡，因为Mr.Chen有着自觉的节能意识，她希望在耗能总数最少的情况下将所有的灯关掉。 Mr.Chen因为太累了，所以只能以1m/s的速度行走。关灯不需要花费额外的时间，因为当她通过时就能将灯关掉。 **编写程序，计算在给定路灯设置，灯泡功率以及Mr.Chen的起始位置的情况下关掉所有的灯需耗费的最小能量。**

【输入格式】 第一行包含一个整数N， $2 \leq N \leq 1000$ ，表示该村庄路灯的数量。

第二行包含一个整数V， $1 \leq V \leq N$ ，表示Mr.Chen开始关灯的路灯号码。

接下来的N行中，每行包含两个用空格隔开的整数D和W，用来描述每盏灯的参数，其中 $0 \leq D \leq 1000$ ， $0 \leq W \leq 1000$ 。D表示该路灯与村庄开始处的距离(用米为单位来表示)，W表示灯泡的功率，即在每秒种该灯泡所消耗的能量数。路灯是按顺序给定的。

【输出格式】 第一行即唯一的一行应包含一个整数，即消耗能量之和的最小值。注意结果小超过1,000,000,000。

区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

【题目07】关灯

在某些区间DP问题中，当前决策可能会影响未来行动的费用。若当前决策对未来的影响只与当前决策有关，则可将对未来费用的影响算作当前决策的费用；若当前决策对未来的影响不仅与当前决策有关，还与未来情况有关，则可新增状态假设未来情况，待到未来决策时使用假设的状态。

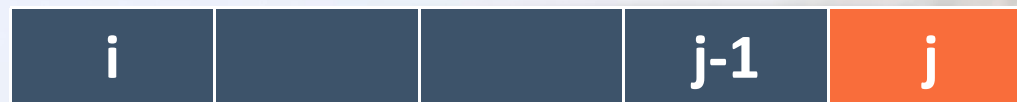
区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

【题目07】关灯

分析：对于 $[i, j]$ 来说，关闭区间的灯之后，有两种情况。在区间左侧或右侧
显然， $f[i][j]$ 只与 $f[i+1][j], f[i][j-1]$ 有关。

对于固定区间区分左右侧，可令 $f[i][j][0]$ 表示在区间左侧， $f[i][j][1]$ 表示在区间右侧。



$$f[i][j][0] = \min\{f[i+1][j][0] + (1[i+1] - 1[i]) * p[i+1][j] \\ f[i+1][j][1] + (1[j] - 1[i]) * p[i+1][j]\}$$

$$f[i][j][1] = \min\{f[i][j-1][1] + (1[j] - 1[j-1]) * p[i][j-1] \\ f[i][j-1][0] + (1[j] - 1[i]) * p[i][j-1]\}$$

区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

【题目07】 [USACO04OPEN] Turning in Homework G

【题目描述】 贝茜有 C ($1 \leq C \leq 1000$) 门科目的作业要上交，之后她要去坐巴士和奶牛同学回家。每门科目的老师所在的教室排列在一条长为 H ($1 \leq H \leq 1000$) 的走廊上，他们只在课后接收作业，交作业不需要时间。贝茜现在在位置0，她会告诉你每个教室所在的位置，以及走廊出口的位置。她每走1个单位的路程，就要用1分钟。她希望你计算最快多久以后她能交完作业并到达出口。

【输入格式】

第一行：三个整数 C, H 和 $B, 1 \leq C \leq 1000, 1 \leq H \leq 1000, 0 \leq B \leq H$ 。

第二行到 $C+1$ 行，第 $i+1$ 行有两个整数 X_i 和 $T_i, 0 \leq X_i \leq H$ 和 $0 \leq T_i \leq 10000$ 。

B 表示车站位置。

X_i 表示第 i 份作业应该在 X_i 这个位置交。

T_i 表示这个位置（教室）的这个科目老师（也就是收这份作业的老师）下课的时间。

【输出格式】

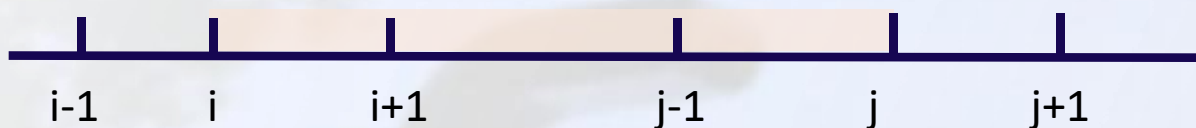
单个整数，表示贝西交完作业后走到车站的最短时间

区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

【题目07】 [USACO04OPEN] Turning in Homework G

【解析】 本题的贡献包含两部分：走路的时间+等待下课的时间



【思考1】 区间 $[i, j]$ 是由哪个区间推导得到， $[i-1, j]$ 、 $[i+1, j]$ 、 $[i, j-1]$ 还是 $[i, j+1]$ ？为什么？

设 $dp[i][j][0]$ 表示当前人在区间 $[i, j]$ 的左端点，且区间 $[i+1, j]$ 的教室没有交作业。

设 $dp[i][j][1]$ 表示当前人在区间 $[i, j]$ 的右端点，且区间 $[i, j-1]$ 的教室没有交作业。

【思考2】 $dp[i][j][0]$ ， $dp[i][j][1]$ 转移方程该如何写？

$dp[i][j][0] = \min(dp[i][j][0], \max(dp[i-1][j][0] + a[i].x - a[i-1].x, a[i].t));$

$dp[i][j][0] = \min(dp[i][j][0], \max(dp[i][j+1][1] + a[j+1].x - a[i].x, a[i].t));$

$dp[i][j][1] = \min(dp[i][j][1], \max(dp[i][j+1][1] + a[j+1].x - a[j].x, a[j].t));$

$dp[i][j][1] = \min(dp[i][j][1], \max(dp[i-1][j][0] + a[j].x - a[i-1].x, a[j].t));$

区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

【题目08】方块消除（题目来源：洛谷P2135）

【问题描述】 Jimmy最近迷上了一款叫做方块消除的游戏。游戏规则如下：n个带颜色方格排成一列，每种颜色用一个整数表示，相同颜色的方块可连成一个区域（如果两个相邻方块颜色相同，则这两个方块可连成一个区域）。

为简化题目，将连起来的同一颜色方块的数目用一个数表示。

例如，9 122233331可表示为

4 //区域数目

1 2 3 1 //每个区域的方块颜色

1 3 4 1 //每个区域的方块数目

游戏时，Jimmy可以任选一个区域消去。假设这个区域包含的方块数为x，Jimmy将得到 x^2 个分值。某个区域的方块消去之后，其右边的所有区域就会向左移动与左边的区域拼接在一起。Jimmy希望你能找出一个最佳消除方案使得得分最高，你能帮助他吗？

【输入格式】 输入包含3行，第一行为一个整数n，代表方块区域的数目。

第二行包含n个整数，第i个数 c_i 代表每个区域的方块颜色。

第三行包含n个整数，第i个数 b_i 代表每个区域的方块个数。

【输出格式】 输出一个整数，代表方块消除的最高得分。

样例输入	样例输出
4 1 2 3 1 1 3 4 1	29

区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

【题目08】方块消除（题目来源：洛谷P2135）

每消除一个区域的方块，就会得到 b_i^2 的分数，假设 $f[i][j]$ 表示消除区间 $[i,j]$ 的方块的最大得分。对于区域 j ，有两种决策：

【决策1】：消除区域 j ，那么 $f[i][j]=f[i][j-1]+b_j^2$ ；

【决策2】：暂时不消除区域 j ，与之前的某个同色区域连在一起消除。对于这种决策，我们需要清楚区间 $[i,j-1]$ 中每个区域的消除情况，但若在状态设计时同时维护每个区域的消除情况，状态数过多。

决策2就是当前决策会对未来行动的费用产生影响的情况（当然也可看作过去决策对当前费用产生影响，这两者是等价的）。与关灯问题不同之处在于，关灯问题中当前决策对未来行动的费用产生的影响与未来情况无关，所以未来的费用可以在当前决策时一起计算；本问题中当前决策对未来行动的影响还与未来情况有关（即与未来的多少个区域连在一起），因此，我们可以在状态中开辟新的维度来假设未来情况。

设 $f[i][j][k]$ 表示消除区间 $[i,j]$ 的方块且区域 j 之后（未来）有 k 个与区域 j 同色的方块的最大得分。对于区域 j ，有两种决策：

决策1：消除区域 j 和与之连接的 k 个同色方块，那么 $f[i][j][k]=f[i-1][j-1][0]+(b_j+k)^2$ 。

决策2：暂时不消除区域 j ，与之前的某个同色区域连在一起消除。假设区间 $[i,j-1]$ 中与区域 j 同色的区域在位置 p ，那么 $f[i][j][k]=f[i][p][k+b_j]+f[p+1][j-1][0]$ 。

综上， $f[i][j][k]=\max(f[i-1][j-1][0]+(b_j+k)^2, f[i][p][k+b_j]+f[p+1][j-1][0])$ 。

边界为 $f[i][j][k]=0$ ， $f[i][i][k]=(b_i+k)^2$ ，即消除区间 $[i,i]$ 的方块且区域 i 之后有 k 个与区域 i 同色的方案的最大得分为 $(b_i+k)^2$ 。

区间DP应用技巧

应用技巧4：增设DP状态，延伸决策效应。

//f[i][j][k]代表消除区间[i,j]的方块且区域j之后（未来）有k个与区域j同色的方块的最大得分

```
for(int len=1;len<=n;len++){//枚举阶段
    for(int i=1;i+len-1<=n;i++){//枚举状态
        int j=i+len-1;
        for(int k=0;k<=s;k++){//决策1
            f[i][j][k]=max(f[i][j][k],f[i][j-1][0]+(b[j]+k)*(b[j]+k));
        }
        for(int p=i;p<j-1;p++){//决策2
            if(c[p]==c[j]){
                for(int k=0;k<=s;k++){
                    f[i][j][k]=max(f[i][j][k],f[i][p][k+b[j]]+f[p+1][j-1][0]);
                }
            }
        }
    }
}
```

经典题目讲解

【题目09】添加括号 (题目来源: P2308)

【问题描述】 给定一个正整数序列 $a(1), a(2), \dots, a(n), (1 \leq n \leq 20)$ 不改变序列中每个元素在序列中的位置, 把它们相加, 并用括号记每次加法所得的和, 称为中间和。例如:

给出序列是4, 1, 2, 3。

第一种添括号方法: $((4+1)+(2+3))=((5)+(5))=(10)$

有三个中间和是5, 5, 10, 它们之和为: $5+5+10=20$

第二种添括号方法: $(4+((1+2)+3))=(4+((3)+3))=(4+(6))=(10)$

中间和是3, 6, 10, 它们之和为19。

现在要添上 $n-1$ 对括号, 加法运算依括号顺序进行, 得到 $n-1$ 个中间和, 求出使中间和之和最小的添括号方法。

经典题目讲解

【题目09】添加括号 (题目来源: P2308)

【输入格式】 共两行。

第一行, 为整数 n 。($1 < n \leq 20$)

第二行, 为 $a(1), a(2), \dots, a(n)$ 这 n 个正整数, 每个数字不超过100。

【输出格式】 输出3行。

第一行, 为添加括号的方法。

第二行, 为最终的中间和之和。

第三行, 为 $n-1$ 个中间和, 按照从里到外, 从左到右的顺序输出。

【输入样例】

4
4 1 2 3

【输出样例】

(4+((1+2)+3))
19
3 6 10

经典题目讲解

【题目09】添加括号 (题目来源: P2308)

分析: 枚举区间, 设置断点求值, 将中间结果记录。

$$f[i][j] = \min(f[i][k] + f[k+1][j] + \text{sum}[j] - \text{sum}[i-1], f[i][j])$$

更新值时记录其断点位置, $\text{ans}[i][j] = k$

借助递归函数处理式子输出与中间和的输出

经典题目讲解

【题目10】监狱（题目来源：洛谷 P1622）

【问题描述】小X的王国中有一个奇怪的监狱，这个监狱一共有P个牢房，这些牢房一字排开，第i个仅挨着第i+1个（最后一个除外），当然第i个也挨着第i-1个（第一个除外），现在牢房正好是满员的。上级下发了一个释放名单，要求每天释放名单上的一个人。这可把看守们吓得不轻，因为看守们知道，现在牢房里的P个人，可以相互之间传话。第i个人可以把话传给第i+1个，当然也能传给第i-1个，并且犯人很乐意把消息传递下去。

如果某个人离开了，那么原来和这个人能说上话的人，都会很气愤，导致他们那天会一直大吼大叫，搞得看守很头疼。如果给这些要发火的人吃上肉，他们就会安静下来。为了河蟹社会，现在看守们想知道，如何安排释放的顺序，才能是的他们消耗的肉钱最少

【输入格式】第一行两个数P和Q，Q表示释放名单上的人数；第二行Q个数，表示要释放哪些人。

【输出格式】仅一行，表示最少要给多少人次送肉吃。

经典题目讲解

【题目10】监狱（题目来源：洛谷 P1622）

【输入样例】 【输出样例】

20 3 35

3 6 14

【样例解释】：先放14号犯人，给19个人肉吃，再放6号犯人，给12个人肉吃，最后放3号，给4个人肉吃，一共35个。

【数据范围】

对于50%的数据： $1 \leq P \leq 100$; $1 \leq Q \leq 5$;

对于100%的数据： $1 \leq P \leq 1000$; $1 \leq Q \leq 100$. $Q \leq P$,

经典题目讲解

【题目10】监狱（题目来源：洛谷 P1622）

【分析】将 n 个人分成 $m+1$ 个区间，每个将要放出的囚犯是一个个的断点，区间内存储的是前面所有的不会放出的囚犯的数量。

阶段：每个区间的长度，即 len

状态：枚举起点的位置，也就是每个区间相当于某一堆石子，

决策：枚举 k 断点

注意区间的划分不是 $1-n$ 。而是 $1-m$ 。重点解决贡献值的计算。

经典题目讲解

【题目10】监狱（题目来源：洛谷 P1622）

【分析】区间dp的套路：

设 $f[i][j]$ 为区间释放 $i \sim j$ 号囚犯所需最少的肉（注意， i, j 不是牢房编号，是释放的囚犯编号，也就是下面的 $sum[i]$ 数组）

$$f[i][j] = \min\{f[i][j], f[i][k-1] + f[k+1][j] + sum[j+1] - sum[i-1] - 1 - 1\}$$

用 $sum[j+1] - sum[i-1] - 1$ 表示 i 到 j 之间要喂多少块肉，然后枚举断点 k ，也就是那个要被释放的人，再-1是因为这个人被释放了就不需要再给肉了。

经典题目讲解

【题目11】数字游戏 (NOIP 2003)

【问题描述】 丁丁最近沉迷于一个数字游戏之中。这个游戏看似简单，但丁丁在研究了许多天之后却发觉原来在简单的规则下想要赢得这个游戏并不那么容易。游戏是这样的，在你面前有一圈整数（一共n个），你要按顺序将其分为m个部分，各部分内的数字相加，相加所得的m个结果对10取模后再相乘，最终得到一个数k。游戏的要求是使你所得的k最大或者最小。

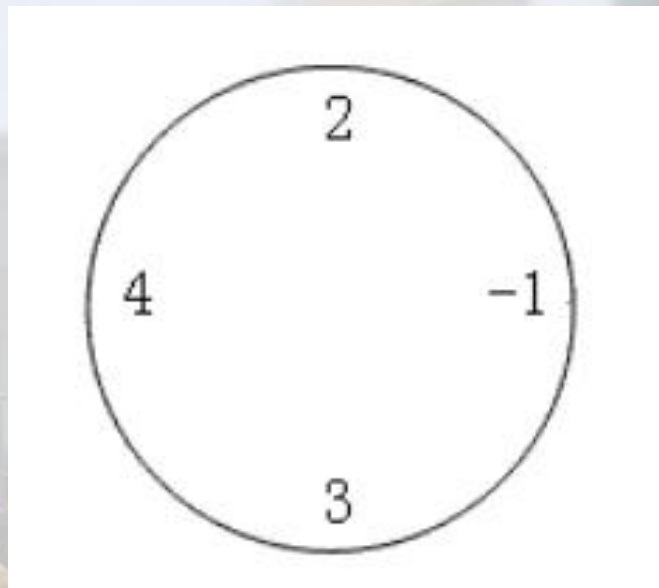
例如，对于下面这圈数字（ $n=4, m=2$ ）：

要求最小值时， $((2-1)\bmod 10) \times ((4+3)\bmod 10) = 1 \times 7 = 7$

要求最大值时， $((2+4+3)\bmod 10) \times (-1 \bmod 10) = 9 \times 9 = 81$

特别值得注意的是，无论是负数还是正数，对10取模的结果均为非负值。

丁丁请你编写程序帮他赢得这个游戏。



经典题目讲解

【题目11】数字游戏 (NOIP 2003)

【输入格式】

输入文件第一行有两个整数， $n(1 \leq n \leq 50)$ 和 $m(1 \leq m \leq 9)$ 。以下 n 行每行有个整数，其绝对值 $\leq 10^4$ ，按顺序给出圈中的数字，首尾相接。

【输出格式】

输出文件有2行，各包含1个非负整数。第1行是你程序得到的最小值，第2行是最大值。

【样例输入】

4 2
4
3
-1
2

【样例输出】

7
81

经典题目讲解

【题目11】数字游戏 (NOIP 2003)

◆断环成链

断环成链+前缀和+区间dp

◆前缀和求解

DP[L][R][k]表示区间L到R内分成k部分的最大值或最小值

比常规的操作多了一个维度，枚举是注意变量范围

```
for(int k=2;k<=m;k++)
    for(int len=k;len<=n;len++)//每个部分起码一个数，所以len要>=k
        for(int l=1;l<=2*n-len+1;l++)
        {
            int r=l+len-1;
            for(int j=l+k-2;j<r;j++)
            {
                dpMax[l][r][k]=max(dpMax[l][j][k-1]*dpMax[j+1][r][1],dpMax[l][r][k]);
                dpMin[l][r][k]=min(dpMin[l][j][k-1]*dpMin[j+1][r][1],dpMin[l][r][k]);
            }
        }
```


经典题目讲解

【题目12】统计单词个数 (NOIP2001)

【题目描述】 给出一个长度不超过 200 的由小写英文字母组成的字母串（该字串以每行 20 个字母的方式输入，且保证每行一定为 20 个）。要求将此字母串分成 k 份，且每份中包含的单词个数加起来总数最大。

每份中包含的单词可以部分重叠。当选用一个单词之后，其第一个字母不能再用。例如字符串 this 中可包含 this 和 is，选用 this 之后就不能包含 th。

单词在给出的一个不超过 6 个单词的字典中。

要求输出最大的个数。

【输入格式】

每组的第一行有两个正整数 p, k 。 p 表示字串的行数， k 表示分为 k 个部分。

接下来的 p 行，每行均有 20 个字符。

再接下来有一个正整数 s ，表示字典中单词个数。 接下来的 s 行，每行均有一个单词。

【输出格式】 1 个整数，分别对应每组测试数据的相应结果。

【样例输入】

```
1 3
thisisabookyouareaoh
4
is
a
ok
sab
```

【样例输出】

7

经典题目讲解

【题目12】统计单词个数 (NOIP2001)

【样例输入】

1 3
thisisabookyouareaoh
4
is
a
ok
sab

【样例输出】

7

【数据范围】 对于 100%的数据， $2 \leq k \leq 40$ ， $1 \leq s \leq 6$ 。

【样例解释】 划分方案为：this / isabookyoua / reaoh

1 + 5 + 1

经典题目讲解

【题目12】统计单词个数 (NOIP2001)

设 $dp[i][k]$ 表示0— i 区间内有 k 个间隔时单词的个数

$$dp[i][k] = \max\{dp[i][k], dp[j][k-1] + g[j+1][i]\}$$

答案为: $dp[n][k]$ (其中 $n=p*20$)

接下来预处理 $g[i][j]$, 即 i 到 j 区间内有多少个单词。(重点和难点)

经典题目讲解

【题目12】统计单词个数 (NOIP2001)

处理贡献值

```
for(int i=1;i<=n;i++)
    for(int j=1;j<=s;j++){
        int t=0;
        while(str[i+t]==word[j][t]&&str[i+t]!='\0'&&word[j][t]!='\0')
            t++;
        if(word[j][t]=='\0') pos[i]=min(pos[i],t);
    }
```

```
for(int i=1;i<=n;i++)
    for(int j=i;j<=n;j++)
        for(int t=i;t<=j;t++)
            if(pos[t]>0 && t+pos[t]-1<=j) g[i][j]++;
```


经典题目讲解

【题目13】涂色（CQOI2007）

【题目描述】假设你有一条长度为 5 的木板，初始时没有涂过任何颜色。你希望把它的 5 个单位长度分别涂上红、绿、蓝、绿、红色，用一个长度为 5 的字符串表示这个目标：RGBGR。每次你可以把一段连续的木板涂成一个给定的颜色，后涂的颜色覆盖先涂的颜色。例如第一次把木板涂成 RRRRR，第二次涂成 RGGGR，第三次涂成 RGBGR，达到目标。用尽量少的涂色次数达到目标。

【输入格式】

输入仅一行，包含一个长度为 n 的字符串，即涂色目标。字符串中的每个字符都是一个大写字母，不同的字母代表不同颜色，相同的字母代表相同颜色。

【输出格式】

仅一行，包含一个数，即最少的涂色次数。

【输入样例1】 AAAAA

【输出样例1】 1

【输入样例2】 RGBGR

【输出样例2】 3

经典题目讲解

【题目13】涂色（CQOI2007）

设 $dp[i][j]$ 字符串中第 i 个字符到第 j 个字符涂好需要最少的次数

当 $i=j$ 时， $dp[i][j]=1$ (表示单个区间)

当 $i \neq j$ 时，若 $s[i]==s[j]$ 可以一次涂多格

$$dp[i][j] = \min\{dp[i+1][j], dp[i][j-1]\}$$

若 $s[i] \neq s[j]$ 则要分成两段来处理，枚举断点 k

$$dp[i][j] = \min\{dp[i][k] + dp[k+1][j], dp[i][j]\}$$

经典题目讲解

总结

枚举区间长度



枚举区间



枚举断点